

File: scheduler.py

```
#!/usr/bin/env python
```

```
import sys
from optparse import OptionParser
import random
```

```
parser = OptionParser()
parser.add_option("<s>", "<--seed>", default=0, help="the random seed",
                action="store", type="int", dest="seed")
parser.add_option("<j>", "<--jobs>", default=3, help="number of jobs in the system",
                action="store", type="int", dest="jobs")
parser.add_option("<l>", "<--jlist>", default="", help="instead of random jobs,
                action="store", type="string", dest="jlist")
parser.add_option("<m>", "<--maxlen>", default=10, help="max length of job",
                action="store", type="int", dest="maxlen")
parser.add_option("<p>", "<--policy>", default="FIFO", help="sched policy to use",
                action="store", type="string", dest="policy")
parser.add_option("<q>", "<--quantum>", help="length of time slice for RR policy",
                action="store", type="int", dest="quantum")
parser.add_option("<c>", help="compute answers for me", action="store_true", default=False)
```

```
(options, args) = parser.parse_args()
```

```
random.seed(options.seed)
```

```
print "ARG policy", options.policy
if options.jlist == "":
    print "ARG jobs", options.jobs
    print "ARG maxlen", options.maxlen
    print "ARG seed", options.seed
else:
    print "ARG jlist", options.jlist
```

```
print "
```

```
print "Here is the job list, with the run time of each job: "
```

```
import operator
```

```
joblist = []
if options.jlist == "":
    for jobnum in range(0, options.jobs):
        runtime = int(options.maxlen * random.random()) + 1
        joblist.append([jobnum, runtime])
        print " Job", jobnum, " ( length = " + str(runtime) + " )"
else:
    jobnum = 0
    for runtime in options.jlist.split(" "):
        joblist.append([jobnum, float(runtime)])
        jobnum += 1
    for job in joblist:
        print " Job", job[0], " ( length = " + str(job[1]) + " )"
print "\n"
```

```
if options.solve == True:
    print "** Solutions **\n"
    if options.policy == "SJF":
        joblist = sorted(joblist, key=operator.itemgetter(1))
        options.policy = "FIFO"

    if options.policy == "FIFO":
        thetime = 0
        print "Execution trace:"
        for job in joblist:
            print " [ time %3d ] Run job %d for %.2f secs ( DONE at %.2f )" % (thetime, job[0], job[1], thetime + job[1])
            thetime += job[1]

    print "\nFinal statistics:"
```

```

t      = 0.0
count = 0
turnaroundSum = 0.0
waitSum      = 0.0
responseSum   = 0.0
for tmp in joblist:
    jobnum = tmp[0]
    runtime = tmp[1]

    response = t
    turnaround = t + runtime
    wait      = t
    print &#x27; Job %3d -- Response: %3.2f Turnaround %3.2f Wait %3.2f&#x27; % (jobnum, response, turnaround)
    responseSum += response
    turnaroundSum += turnaround
    waitSum      += wait
    t += runtime
    count = count + 1
print &#x27;\n Average -- Response: %3.2f Turnaround %3.2f Wait %3.2f\n&#x27; % (responseSum/count, turnaroundSum/count, waitSum/count)

if options.policy == &#x27;RR&#x27;:
    print &#x27;Execution trace:&#x27;
    turnaround = {}
    response = {}
    lastran = {}
    wait = {}
    quantum = float(options.quantum)
    jobcount = len(joblist)
    for i in range(0, jobcount):
        lastran[i] = 0.0
        wait[i] = 0.0
        turnaround[i] = 0.0
        response[i] = -1

    runlist = []
    for e in joblist:
        runlist.append(e)

    thetime = 0.0
    while jobcount > 0:
        # print &#x27;%d jobs remaining&#x27; % jobcount
        job = runlist.pop(0)
        jobnum = job[0]
        runtime = float(job[1])
        if response[jobnum] == -1:
            response[jobnum] = thetime
            currwait = thetime - lastran[jobnum]
            wait[jobnum] += currwait
            if runtime > quantum:
                runtime -= quantum
                ranfor = quantum
                print &#x27; [ time %3d ] Run job %3d for %.2f secs&#x27; % (thetime, jobnum, ranfor)
                runlist.append([jobnum, runtime])
            else:
                ranfor = runtime;
                print &#x27; [ time %3d ] Run job %3d for %.2f secs ( DONE at %.2f )&#x27; % (thetime, jobnum, ranfor)
                turnaround[jobnum] = thetime + ranfor
                jobcount -= 1
            thetime += ranfor
            lastran[jobnum] = thetime

    print &#x27;\nFinal statistics:&#x27;
    turnaroundSum = 0.0
    waitSum      = 0.0
    responseSum   = 0.0
    for i in range(0, len(joblist)):
        turnaroundSum += turnaround[i]
        responseSum += response[i]
        waitSum += wait[i]
        print &#x27; Job %3d -- Response: %3.2f Turnaround %3.2f Wait %3.2f&#x27; % (i, response[i], turnaround[i])

```

```

count = len(joblist)

print '\n Average -- Response: %.2f Turnaround %.2f Wait %.2f\n' % (responseSum/count, t

if options.policy != 'FIFO' and options.policy != 'SJF' and options.policy != 'RR&
print '\nError: Policy', options.policy, 'is not available.\n';
sys.exit(0)
else:
print '\nCompute the turnaround time, response time, and wait time for each job.\n';
print '\nWhen you are done, run this program again, with the same arguments,\n';
print '\nbut with -c, which will thus provide you with the answers. You can use\n';
print '\n-s <somenumber> or your own job list (-l 10,15,20 for example)\n';
print '\nto generate different problems for yourself.\n';
print '\n';

```